NOVA IMS INFORMATION MANAGEMENT SCHOOL

How do you know it works?

Evaluating GenAI applications — part one of two

Session 5 · Evaluation I

Hugo Silva, September 2026

Tonight

Four weeks building. Two weeks judging. Tonight, with no model's help.

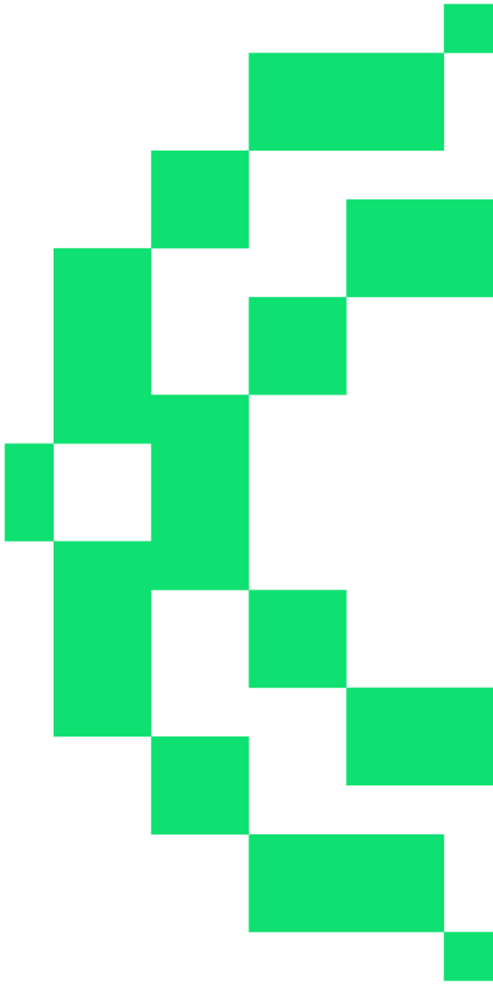
1 · Why one run is not a measurement.

2 · Look before you measure. Error analysis, done properly.

3 · Build a classifier, then score it against your own labels.

4 · The metric is a decision about which mistake costs more.

Then the project. Next week: agents judging agents.



01.
Why

One run is not a measurement

One run is not a measurement

Session 1 showed you this. Here is what it costs you.

SESSION 1, DEMO 3 — ONE PROMPT, FIVE RUNS

the same prompt, five times ➡

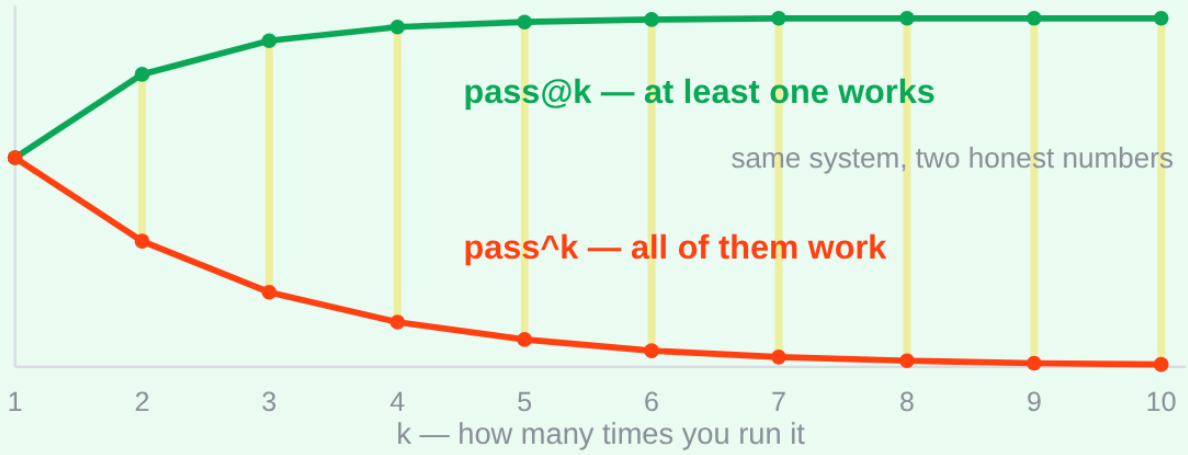
- run 1 ...refund is due.
- run 2 ...not eligible.
- run 3 ...refund is due.
- run 4 ...refund is due.
- run 5 ...escalate to billing.

👉 **this is the one you demoed**

60%

what your users get

THE SAME FACT, WITH ARITHMETIC



pass@k if a human checks the output. pass^k if nobody does.

Session 1, demo 3 — sampling. Anthropic Engineering, "Demystifying Evals for AI Agents" (9 Jan 2026).

What an eval actually is

Three parts, and two of them are not the model

1 · TASK

“a single test with defined inputs and success criteria”

```
ticket #04 → expected: billing
```

2 · TRANSCRIPT

“the complete record of a trial, including outputs, tool calls, reasoning, intermediate results”

where the work is — and nobody opens it

3 · GRADER

“logic that scores some aspect of the agent's performance”

code == | a model | a person

You choose the grader.

The other two you collect.

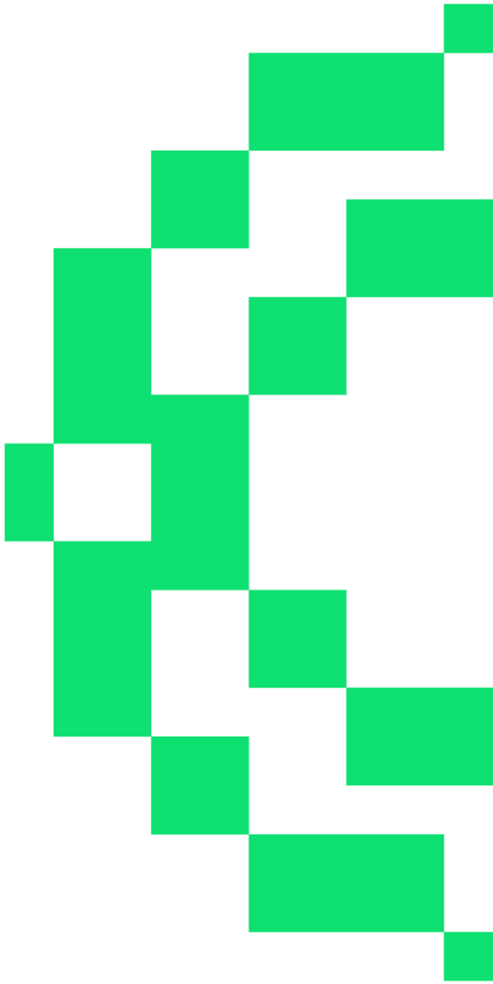
Most rooms hear “evaluation” and picture the score. The score is the smallest of the three parts, and the transcript is the one nobody reads.

Definitions verbatim — Anthropic Engineering (9 Jan 2026). These three words are used unqualified for the rest of both sessions.

Error analysis is the most important activity in evals.

Hamel Husain, LLM Evals FAQ

From the same FAQ: “We've spent 60–80% of our development time on error analysis and evaluation.”



02.
Look

Before you measure anything

Open coding, in three passes

Read, name what went wrong, cluster. Counting comes last.

PASS 1 · READ AND ANNOTATE

Twenty transcripts. One free-text sentence beside each.

#01 *fine*
#06 *said high urgency because they typed in capitals*
#11 *the quote is nowhere in the ticket*

No category list yet. *That is the discipline.*

➡

PASS 2 · CLUSTER

Group the sentences. Name each family.

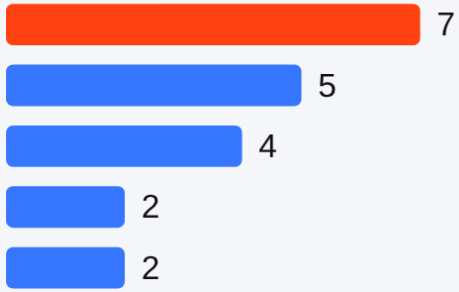
urgency inflated by tone | wrong category | quote not in source | should have abstained | two categories at once

A taxonomy that did not exist an hour ago.

➡

PASS 3 · COUNT

Only now does a number mean anything.



An ordered list of what to fix.

BINARY, AND CRITIQUE FIRST

Three people score the same output **3, 4 and 2**. Mean 3.0 — and nobody agrees on why. Asked **does it pass?**, all three say **fail**, and their three reasons differ visibly. That is the useful part.

"Binary evaluations force clearer thinking and more consistent labeling."

WHEN TO STOP

Stop when **~20 traces** turn up no new category.
Target **~100** for a system in production.

A number you cannot act on is not a measurement — which is why nobody's first eval is an off-the-shelf coherence score.

Husain & Shankar, LLM Evals FAQ. The method is open coding, borrowed from qualitative social science (Strauss & Corbin, 1990).

Demo 1 — Read twenty outputs

Twelve minutes. Setup: open a sheet, make a copy.

LIVE DEMO

Kestrel sheet, tab “read me”

1. Open the sheet from Moodle and make a copy. Twenty tickets, and what the classifier said about each: category, urgency, evidence quote.
2. Read the first ten. Write one sentence per row in the notes column — “fine”, or what is wrong, in your own words. Do not invent a category list.
3. Compare with your neighbour on tickets 7 and 13, or any pair of tickets. Where you disagree, do not settle it — write down why.
4. Read your own twenty sentences and group them. Four or five families. Name each one so a colleague would understand it.
5. Count them. The largest family is where your next hour goes — and only now does a metric make any sense.

WHAT TO WATCH FOR

- The names came out of your data, not a blog post. Yours will not match your neighbour's, and that is the point.
- Step 3's disagreements are the most valuable thing you produce: nobody had decided what “correct” meant there.
- You will want to skip to step 5. The count is worthless if steps 2–4 were rushed.
- Twenty finds your biggest failure. It does not make the proportions trustworthy.

No model runs in this demo. Everything you need is already in the sheet — which is also true of the first day of most real eval projects.

From failure mode to criterion

You just produced the first. The second is what you measure.

FAILURE MODE

a specific, repeatable pattern of incorrect, unsafe or undesirable behaviour.

What goes wrong. You find these by **reading**.

becomes ➡

EVALUATION CRITERION

a defined dimension, standard or metric used to measure a specific quality of the output.

What you check, every time. You **write** these.

FAILURE MODE FOUND	N	THE CRITERION IT BECOMES	GRADER
urgency inflated by emotional wording	7	urgency matches the policy definition	judge
the wrong category when two could apply	5	recall on each category, scored separately	code
evidence quote not present in the ticket	4	the quote is a substring of the ticket	code
answered when it should have abstained	2	the category is in the allowed list	code
more than one category, or reasoning mixed in	2	exactly one label, and nothing else in the field	code

13 of 20 failures need a string comparison, not a model. A criterion with no failure behind it is decoration.

Counts from the frozen demo set. Your taxonomy will differ; the shape of the finding usually does not.

03. Measure

Build the classifier, then score it

Labels are definitions, not names

A model cannot read your mind about where billing stops.

WHAT EVERYONE WRITES

`billing`

`account_access`

`bug_report`

`feature_request`

`other`

Five words. Three annotators produce three different label sets from them — and the model is a fourth annotator who never got a training day.

WHAT A LABEL SET HAS TO CONTAIN

`billing` — the customer disputes, questions or wants to change what they were charged.

include: double charges, refunds, plan changes, invoices · **exclude:** *cannot log in because the plan lapsed* → that is account_access. The money is context; the blocked access is the problem.

`account_access` — the customer cannot get into the product, or into part of it.

include: password, MFA, lockout, seats, permissions · **exclude:** *can log in, dislikes the price* → that is billing.

TIE-BREAKER, ONCE FOR THE WHOLE SET

When two apply, label the thing the customer **wants fixed first**.

THE ESCAPE HATCH

`other` is **an answer, not a failure**. Do not stretch a label to avoid it.

Four parts per label: a definition · an inclusion · a boundary against its nearest neighbour · a tie-breaker.

The boundary rules come out of demo 1: every confusion pair you found is a boundary nobody had written down.

The classifier prompt, in full

Kestrel's support chatbot, as it would really be written

```
You classify inbound support messages for Kestrel Software.          ①
Return the intent, the urgency, and the sentence you based it on.

LABELS — use these definitions, not your own:                        ②
  billing          the customer disputes, questions or wants to
                   change what they were charged.
                   NOT: cannot log in because the plan lapsed.
  account_access   the customer cannot get into the product.
                   NOT: can log in, dislikes the price.
  bug_report       behaves differently from what is documented.
  feature_request  something absent that the customer wants.
  other            none of the above. This is a valid answer.

WHEN TWO APPLY: label what the customer wants fixed first.           ③

URGENCY: high only if the customer is blocked from working now.      ④
  Tone, capitals and politeness are not evidence of urgency.

EVIDENCE: quote one sentence from the message, copied exactly.       ⑤
  If you cannot find one, return "".

Return JSON only:                                                    ⑥
{"intent": ..., "urgency": ..., "evidence": ...}
```

EVERY BLOCK IS A DECISION

① **role**
One line. Who it is for, and what comes back.

② **the label block**
Pasted verbatim from the previous slide — definitions and boundaries, not names.

③ **the tie-breaker**
Stated once for the whole set, so it cannot drift between labels.

④ **a criterion, as an instruction**
Demo 1 found **seven** failures of exactly this kind. This line is the fix.

⑤ **designed to be graded**
Makes the failure machine-checkable: is the quote a substring of the ticket?

⑥ **structured output**
Session 2, move four. Nothing downstream works without it.

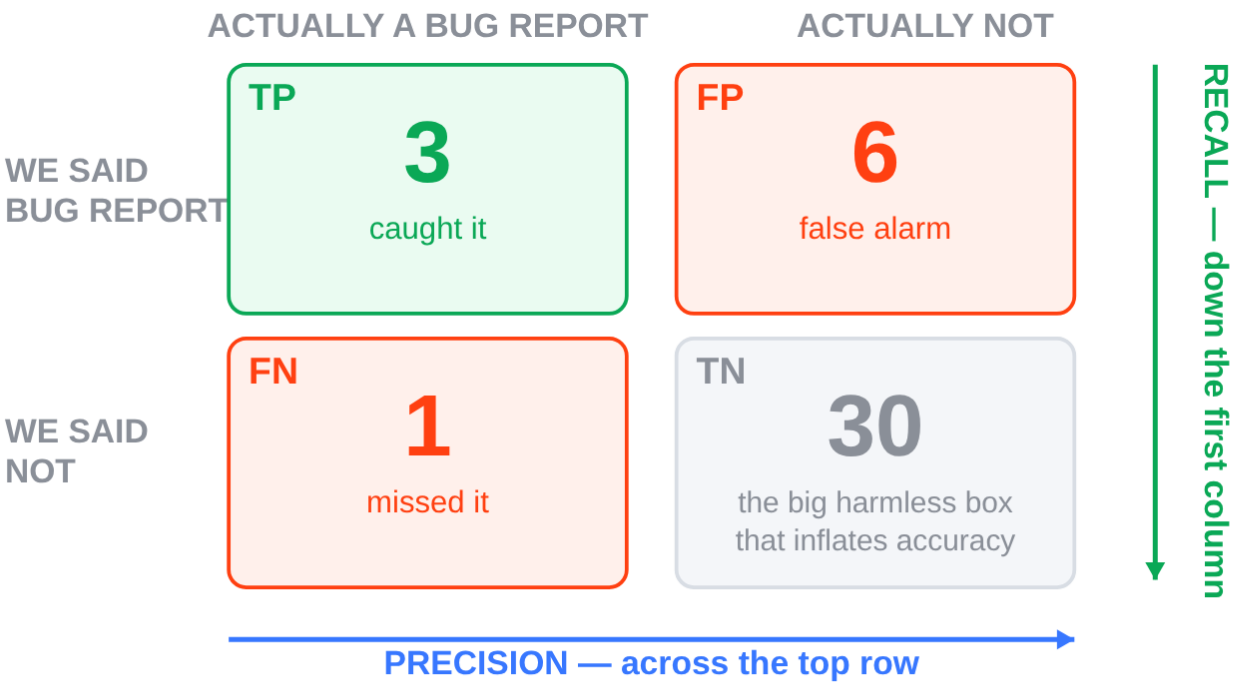
Two worked examples follow as few-shot pairs — one of them the **lapsed-subscription** boundary case.

Every line here is a Session 2 technique. What is new is that you can now measure whether it helped.

Session 2's five moves — role, definitions, few-shot, structured output, an explicit abstain. The urgency line exists because demo 1 found seven failures of exactly that kind.

Four boxes, three ratios

Accuracy, precision and recall, from one table



PRECISION

"Of the ones we flagged, how many were right?"

3 ÷ 9 = **0.33** · the cost is wasted engineering time

RECALL

"Of the ones that mattered, how many did we catch?"

3 ÷ 4 = **0.75** · the cost is a bug nobody saw

AND THE FORMULAS, FOR WHOEVER WANTS THEM

```
precision = TP / (TP + FP)
recall    = TP / (TP + FN)
accuracy  = (TP + TN) / all
```

ACCURACY — THE THIRD RATIO, AND THE ONE THAT LIES

(TP + TN) ÷ all forty = (3 + 30) ÷ 40 = **0.825**. Eighty-two per cent, from a classifier that found three bug reports and invented six. **The big harmless box did the work.**

ONE TABLE PER LABEL

Five labels, five tables — which is why one headline number cannot be honest, and why the next slide exists.

Identical for multi-class and multi-label. In multi-label a ticket simply appears in more than one table.

Macro or micro?

Five tables, one number. Two honest ways to get there.

YOU HAVE FIVE TABLES

One per label, each with its own four boxes.

billing · account_access · bug_report · feature_request · other

Five precisions. Five recalls.

A report needs one number. There is no single correct way to get there — there are two, and they answer different questions.

MACRO — AVERAGE THE SCORES

Work out the score for each label, then take the plain mean. **Every label counts the same, however rare.**

```
macro-P = (P1 + P2 + P3 + P4 + P5) / 5
```

A class with four examples gets the same vote as one with four hundred, so **one broken rare class drags the number down where you can see it**.

MICRO — POOL THE BOXES

Add every TP, FP and FN across all five labels first, then compute once. **Every decision counts the same.**

```
micro-P = ΣTP / (ΣTP + ΣFP)
```

The common labels supply most of the decisions, so they decide the number — and **a broken rare class can disappear inside it**.

WHICH ONE

Macro when the rare class is the point — fraud, outages, bug reports, anything you built the system to catch. **Micro when every decision costs the same** and what you care about is total throughput.

AND ALWAYS

Say which one you used. **An average with no name is not a number anyone can read.**

Neither is more correct than the other. Reporting an average without naming it is the only wrong answer.

When macro and micro disagree

Same model, same metric. Only the mix of tickets changed.

TONIGHT — THE FORTY TICKETS IN YOUR SHEET

LABEL	TP	FP	PRECISION
billing	11	0	**1.00**
account_access	10	0	**1.00**
bug_report	3	6	**0.33**
feature_request	8	0	**1.00**
other	5	1	**0.83**
all five	37	7	

MACRO
4.16 / 5
0.83

MICRO
37 / 44
0.84

One point apart. Either would do, and nobody would notice which you quoted.

A MONTH LATER — TWO THOUSAND TICKETS, SAME CLASSIFIER

LABEL	TP	FP	PRECISION
billing	679	21	**0.97**
account_access	614	26	**0.96**
bug_report	40	80	**0.33**
feature_request	399	21	**0.95**
other	180	20	**0.90**
all five	1,912	168	

MACRO
4.11 / 5
0.82

MICRO
1,912 / 2,080
0.92

Ten points apart. *Micro puts it on the dashboard as a good month.*

The model is identical in both tables. Only the mix of tickets changed.

bug_report is one label in five either way — but **20% of tonight's predictions** and **6% of the month's**. Macro never notices the mix; micro is made of it.

The left-hand table is the sheet in front of you. The right-hand one is the same classifier on a month of traffic — illustrative, not measured.

Which error can you afford?

Price the mistake first. The metric follows from that.

ROUTING TO THE RIGHT TEAM

Both errors cost the same handful of minutes, and a human re-routes it either way.

the mistake you accept: a few minutes of the wrong person's time

per-class accuracy is fine

FLAGGING A POSSIBLE OUTAGE

A miss is a public incident. A false alarm is a five-minute check by someone who is already awake.

the mistake you accept: false alarms, deliberately and often

recall

AUTO-CLOSING TICKETS

Closing a live problem loses a customer. Leaving one open costs nothing but a queue.

the mistake you accept: a longer queue, forever

precision, aggressively

AND F1?

The compromise between the two. Use it when both errors **genuinely** cost the same — and be able to say out loud why you believe that. On this slide, only the first column qualifies.

TWO WAYS TO REPORT A NUMBER THAT IS NOT A NUMBER

Accuracy on an imbalanced set is a hiding place, not a compromise — 90% by never predicting the rare class at all. **An average with no name** on a multi-label set: say **macro** or **micro**, or you have not reported anything.

You do not pick a metric. You price a mistake, and the metric follows.

The question is never "which metric is correct" but "which of these two mistakes would I rather explain to my manager".

One label, or several?

Real chatbots return a list. That changes the arithmetic.

MULTI-CLASS — EXACTLY ONE LABEL

"I want a refund for last month."

billing

Every row is right or wrong. Accuracy means something.

MULTI-LABEL — ZERO OR MORE

"I was charged twice and now I cannot log in."

billing account_access

***Six of the forty tickets in your sheet are like this**, because real inboxes are.*

1 · ACCURACY NEARLY DIES

Exact-set match scores `{billing}` against `{billing, account_access}` as a **total failure** — the same as `{other}`. Half right scores zero.

2 · YOU SCORE PER LABEL

Precision and recall for **billing**, then for **account_access**, then for each of the rest. **Five tables, not one.**

3 · THEN YOU AVERAGE — AND THAT IS A CHOICE

macro or **micro**, as defined earlier. An average with no name, on a multi-label set, is not a number anyone can read.

Macro when the rare class is the point.

The prompt change is two lines: `return a list and return an empty list if none apply`

Multi-class and multi-label use identical per-class machinery — slide 14. Only the averaging, and the honesty of "accuracy", differ.

The golden set

Your inputs, your labels, agreed before anyone saw a score.

THE FIELD	WHAT IT HAS TO BE	THE MISTAKE PEOPLE MAKE
what is in it	real inputs, not invented ones	synthetic inputs test the app you imagined
who labelled it	named humans, before any model output	labelling after you see the answer is not labelling
how big	start at 20; grow it every time production surprises you	“20-50 simple tasks drawn from real failures is a great start”
frozen and versioned	one file, one version, checked in	a score means nothing if the set moved under it
never in the prompt	not one case, not as a few-shot example	the moment it goes in the prompt it stops being a test

ONE ROW OF A GOLDEN SET, IN FULL

```
id    : KS-0117
input : "Charged twice for March and now the app won't let me in."
labels: ["billing", "account_access"]
urgency: high    agreed_by: HS, MP    date: 2026-10-07
note  : two labels; MP first said billing only — tie-breaker resolved it
```

*The **agreed_by** and **note** fields are the ones people leave out, and the ones you need six weeks later when the score is disputed.*

The weakest part of a golden set is always the labelling.

It is never the model. If two of you disagree on a label, every number downstream is noise — see the appendix on Cohen's kappa.

Anthropic Engineering (Jan 2026): “20-50 simple tasks drawn from real failures is a great start.” Your demo-1 notes are that queue.

The cheapest grader that works

Two graders that run, and the human who writes the answers.

GRADERS — THESE RUN ON EVERY BUILD, FOREVER

CODE
about nothing, per check

"Fast, cheap, objective, reproducible, easy to debug."

catches: format · forbidden words · quote-not-in-source · label not in list · the wrong category

MODEL
one call per check

"Flexible, scalable, captures nuance, handles open-ended tasks."

catches: tone · groundedness · did-it-answer-the-question — **plus the work of proving the judge agrees with you. That is all of next week.**

NOT A GRADER — WHERE THE ANSWERS COME FROM

HUMAN
once, not on every run

"Gold standard quality, matches expert user judgment."

produces: the golden set · the label definitions · next week's judge calibration — **the answers the other two are scored against.** A few dozen cases an evening, and in a running application it is not in the loop at all.

If an assertion or a regular expression catches it, that is the answer.

FROM YOUR OWN TAXONOMY

13 of 20 failures are caught by code alone. The other **seven** need a model — and a human to say whether the model got them right.

Use the cheapest grader that catches it — reach for a costlier one because the cheaper one cannot do the job, never because it sounds more serious.

Grader descriptions verbatim, Anthropic Engineering (Jan 2026); the assertion rule from Husain's evals FAQ.

Demo 2 — Score it properly

Twelve minutes. Setup: the second tab of the same sheet.

LIVE DEMO

Kestrel sheet, tab “score me”

1. Switch to the second tab. Forty tickets, labels agreed in advance, and the predictions already there — you are reading a result, not waiting for one.
2. Read the headline accuracy. Write it down. It looks respectable, and you would put it on a slide.
3. Look at the breakdown by class. bug_report occurs four times in forty. Check how many of those four it found — and how often it cried bug.
4. Read macro and micro side by side. Macro is the lower of the two. Work out which class drags it down, and why micro does not notice.

WHAT TO WATCH FOR

- Step 3 is the whole lesson: the headline says 72.5% and the class you care about scores 0.46. Both are true.
- Precision and recall are not two opinions about quality. They price two different mistakes.
- The fragile part here is the labels, not the model. Disagree on one and everything downstream is noise.
- Nothing needed a model to grade it. Comparing two labels is an equals sign.
- Macro 0.79, micro 0.82 — close, because bug_report is a fifth of the predictions here. On a real month it would not be.

Four tickets in forty are bug reports, and the classifier called nine of them. No headline accuracy will ever show you that.

04.
So what?

What to do on Monday

What to take away

Five sentences you should be able to say to a colleague

- One run is not a measurement. Quote pass@k or pass^k, and say which — they differ by everything.
- Read twenty outputs and name the failures in your own words. Criteria come from failures you actually saw.
- A label is a definition, a boundary and a tie-breaker. Five bare words is not a specification, for a person or a model.
- A golden set is your labels, agreed before anyone saw a score, frozen, and never used as prompt examples.
- Accuracy hides the rare class, and on multi-label it barely works. Score per class, then say macro or micro.

Reading for next week: Zheng et al., MT-Bench (2023) · Es et al., RAGAS (2024) · Husain, "Using LLM-as-a-Judge"

References

Sources used in this session

- Anthropic Engineering (2026). Demystifying Evals for AI Agents (9 Jan 2026).
- Husain, H. & Shankar, S. LLM Evals FAQ — hamel.dev/blog/posts/evals-faq/
- Husain, H. Creating a LLM-as-a-Judge That Drives Business Results — hamel.dev/blog/posts/llm-judge/
- Shankar, S. & Husain, H. (2026). Evals for AI Engineers. O'Reilly (early release).
- Zheng et al. (2023). Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. NeurIPS D&B.
- Es et al. (2024). RAGAS: Automated Evaluation of Retrieval Augmented Generation.
- Yao et al. (2024). τ-bench: A Benchmark for Tool-Agent-User Interaction. Sierra.
- Strauss, A. & Corbin, J. (1990). Basics of Qualitative Research — the origin of open coding.

NOVA IMS INFORMATION MANAGEMENT SCHOOL

See you next week.

NOVA IMS Accreditations and Certifications

eduniversal · QS STARS · iSchools · EMOS · UNIGIS · USGIF · Erasmus Mundus Programme Education and Culture DG · apcer · IQNet

Co-financed by

PORTUGAL 2030 · Lisboa 2030 Programa Regional de Lisboa · Co-funded by the European Union · fct Fundação para a Ciência e a Tecnologia · PRR Plano de Recuperação e Resiliência · República Portuguesa · Financiado pela União Europeia NextGenerationEU

A.
Appendix

Backup slides

Are your labels any good?

Agreeing 80% of the time may be agreeing by accident

WHY RAW AGREEMENT FLATTERS

Two annotators label a set that is **90% other**. They agree **90% of the time** — while guessing. The number says nothing about whether the labels are any good.

WHAT KAPPA IS

The agreement left over once you subtract the agreement two people would reach by chance, given how often each of them uses each label.

$$\kappa = (p_o - p_e) / (1 - p_e)$$

p_o — the share of items they actually labelled the same. **p_e** — the share they would have matched on by chance alone.
1 perfect · **0** no better than chance · **below 0** worse than chance.

```
observed agreement  = 0.90
expected by chance = 0.82
kappa = (0.90 − 0.82) / (1 − 0.82) = 0.44
```

Ninety per cent looked excellent. The labels are barely usable.

RULES OF THUMB — AND THEY ARE ONLY THAT

below 0.4	the labels are not usable
0.4 – 0.6	the **guidelines** are ambiguous, not the annotators careless
above 0.8	strong

WHAT TO DO ABOUT IT

Rewrite the **label definitions** — the four parts from slide 12. A disagreement is a specification bug, and it is fixed in the spec, never by asking people to concentrate harder.

The fix for a low kappa is never “try harder”.

It is to rewrite the definitions — which is exactly what demo 1 step 3 was designed to surface.

Cohen (1960); Landis & Koch (1977) for the bands — which are conventions, not findings.

Why not let it grade itself?

Session 4 said it cannot. Next week says it can. Both hold.

SELF-CORRECTION — FAILS

One call. Its own answer. Its own reasoning. **No external reference.**

Huang et al. (ICLR 2024): reflecting with no external signal often makes reasoning **worse** — a model that got an answer wrong usually cannot tell that it did.

This is why Session 4 told you a coding agent's self-review works (the tests are the external check) and a chatbot told to "double-check your work" does not.

VS

LLM-AS-JUDGE — WORKS, CONDITIONALLY

a separate call
not the same turn reflecting on itself

a different prompt
written to grade, not to answer

an external rubric
criteria it did not invent while answering

calibrated on human labels
you measured the agreement before trusting it

Four bolts hold it together. Remove any one and you are back on the left.

Session 4 said models cannot check themselves. Next week builds one that can.

Both are true, and the four bolts are the whole difference.

Huang et al., "LLMs Cannot Self-Correct Reasoning Yet" (ICLR 2024) · Kamoi et al., TACL 2024: self-correction works when there is a reliable external check.